

MATERIA: INF560 — DESARROLLO WEB BACKEND

PROYECTO FINAL

Sistema Web de Gestión de Proyectos Colaborativos

Documento de Especificación y Evaluación

CONTENIDO DEL DOCUMENTO

1. Información general
2. Competencia y objetivos
3. Descripción del proyecto
4. Stack tecnológico
5. Modelo de datos
6. Roles y permisos
7. Requisitos funcionales
8. Arquitectura de la aplicación
9. Fases de desarrollo
10. Entregables
11. Criterios de evaluación
12. Recomendaciones técnicas

COMPETENCIA DEL PROYECTO

El estudiante diseña e implementa una aplicación web monolítica con el framework Laravel, integrando autenticación basada en sesión, un sistema de control de acceso por roles y permisos, y operaciones CRUD sobre un dominio relacional, aplicando buenas prácticas de validación, autorización y control de versiones para entregar un producto funcional y mantenible.

1 Información general

Campo	Detalle
Asignatura	INF560 — Desarrollo Web Backend (5.º semestre)
Carrera	Ingeniería Informática — Facultad de Ciencias Puras
Institución	Universidad Autónoma Tomás Frías (UATF) — Potosí
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Modalidad	Individual
Arquitectura	Aplicación monolítica (Laravel + Blade)
Ponderación	100 puntos (ver criterios de evaluación)

2 Competencia y objetivos

Objetivo general

Desarrollar una aplicación web monolítica completa con Laravel que gestione un dominio relacional, incorporando autenticación de usuarios, control de acceso por roles y permisos, y operaciones CRUD con validación, demostrando dominio del flujo backend de extremo a extremo.

Objetivos específicos

- Modelar un dominio relacional con relaciones 1:N y N:M usando Eloquent, migraciones, factories y seeders.
- Implementar autenticación por sesión nativa de Laravel (guard web), con protección de rutas mediante middleware y manejo de CSRF.
- Configurar un sistema de roles y permisos con spatie/laravel-permission, diferenciando capacidades por rol.
- Aplicar autorización por pertenencia mediante Policies y reflejar el control de acceso en las vistas Blade.
- Construir interfaces con Blade (layouts, partials, formularios validados, mensajes flash y paginación).
- Versionar el desarrollo con Git, entregando por fases mediante etiquetas (tags).

3 Descripción del proyecto

Se desarrollará «GestorPro», una aplicación web monolítica para la gestión de proyectos colaborativos, inspirada en herramientas como Trello o Jira pero acotada al alcance de un proyecto final. Los equipos crean proyectos, dentro de ellos gestionan tareas con estados y prioridades, y los miembros colaboran mediante comentarios.

La interacción se realiza íntegramente a través de vistas Blade y formularios tradicionales. El énfasis está en el flujo backend: modelado de datos, autenticación, autorización granular y operaciones CRUD robustas. El sistema de roles no es decorativo: cada rol habilita o restringe acciones concretas, lo que permite una evaluación objetiva del control de acceso.

4 Stack tecnológico

Componente	Tecnología / Versión
Framework	Laravel 13
Lenguaje	PHP 8.3 o superior
Base de datos	PostgreSQL
Autenticación	Sesión nativa (Laravel Breeze o login manual)
Roles y permisos	spatie/laravel-permission ^7.0
Vistas	Blade (layouts, components/partials)
Validación	Form Requests
Control de versiones	Git (entrega por fases con tags)

5 Modelo de datos

El dominio mínimo se compone de las siguientes entidades. Se exige al menos una relación 1:N y una relación N:M con datos en la tabla pivote, además del uso de soft deletes.

Entidad	Campos clave	Relaciones
User	name, email, password	N:M con Project (miembros); 1:N con Task (responsable); 1:N con Comment
Project	nombre, descripción, estado, owner_id	1:N con Task; N:M con User (pivote project_role)
Task	título, descripción, estado, prioridad, due_date, assignee_id	N:1 con Project; N:1 con User; 1:N con Comment
Comment	cuerpo, user_id, task_id	N:1 con Task; N:1 con User
Label (opcional)	nombre, color	N:M con Task

PIVOTE CON ROL DE PROYECTO

La relación N:M entre User y Project usa una tabla pivote (project_user) que almacena un campo project_role (por ejemplo: lider, colaborador, invitado), gestionado con attach, detach y updateExistingPivot.

6 Roles y permisos

El control de acceso se implementa con spatie/laravel-permission. Se definen cuatro roles globales con permisos diferenciados:

Rol	Permisos representativos
Administrador	Gestionar usuarios y asignar roles; ver, editar y eliminar cualquier proyecto y tarea del sistema.
Líder de Proyecto	Crear proyectos; gestionar miembros de sus proyectos; crear, asignar y eliminar tareas de sus proyectos.
Colaborador	Ver proyectos en los que es miembro; crear y editar tareas asignadas a él; comentar.

Rol	Permisos representativos
Invitado / Cliente	Acceso de solo lectura a los proyectos en los que fue agregado; comentar.

Permiso por rol vs. autorización por pertenencia

El punto técnico clave es distinguir dos capas: el permiso por rol (por ejemplo, «puede crear proyectos») se resuelve con spatie; mientras que la autorización por pertenencia (por ejemplo, «solo puede editar SUS proyectos») se resuelve con Policies. Ambas capas deben además reflejarse en las vistas, ocultando los controles que el usuario no puede usar.

EJEMPLO — Definición de permisos en el seeder

```
// database/seeder/RolePermissionSeeder.php
$permisos = ['ver proyecto', 'crear proyecto', 'editar proyecto',
    'eliminar proyecto', 'gestionar miembros', 'crear tarea',
    'asignar tarea', 'comentar', 'gestionar usuarios'];

foreach ($permisos as $p) {
    Permission::firstOrCreate(['name' => $p]);
}

$lider = Role::firstOrCreate(['name' => 'lider']);
$lider->givePermissionTo(['crear proyecto', 'editar proyecto',
    'gestionar miembros', 'crear tarea', 'asignar tarea', 'comentar']);
```

EJEMPLO — Policy por pertenencia

```
// app/Policies/ProjectPolicy.php
public function update(User $user, Project $project): bool
{
    return $user->hasRole('admin')
        || ($user->can('editar proyecto')
            && $project->owner_id === $user->id);
}
```

EJEMPLO — Control en la vista Blade

```
{{-- resources/views/projects/show.blade.php --}}
@can('update', $project)
    <a href="{{ route('projects.edit', $project) }}">Editar</a>
@endcan
```

7 Requisitos funcionales

1. Registro e inicio de sesión de usuarios con autenticación por sesión y protección CSRF.
2. Panel privado accesible solo para usuarios autenticados (middleware auth).
3. CRUD de proyectos, respetando permisos por rol y autorización por pertenencia.
4. Gestión de miembros de un proyecto (agregar/quitar) con su rol en el pivote.
5. CRUD de tareas anidadas a un proyecto, con estado, prioridad y responsable.
6. Cambio de estado y reasignación de tareas mediante acciones específicas.
7. Comentarios en tareas, disponibles para miembros e invitados.
8. Área de administración para gestionar usuarios y asignar roles (solo Administrador).
9. Listados con filtros (estado, prioridad) y paginación.
10. Validación de formularios con Form Requests y mensajes flash de resultado.

8 Arquitectura de la aplicación

Rutas y controladores

Se utilizan rutas web con controladores tipo resource, protegidas por middleware de autenticación y de permisos/Policiés.

```
// routes/web.php
Route::middleware('auth')->group(function () {
    Route::resource('projects', ProjectController::class);
    Route::resource('projects.tasks', TaskController::class);
    Route::post('projects/{project}/members', [MemberController::class, 'store']);
    Route::delete('projects/{project}/members/{user}',
[MemberController::class, 'destroy']);
    Route::patch('tasks/{task}/status', [TaskController::class, 'status']);
    Route::patch('tasks/{task}/assign', [TaskController::class, 'assign']);
    Route::post('tasks/{task}/comments', [CommentController::class, 'store']);

    Route::middleware('role:admin')->prefix('admin')->group(function () {
        Route::get('users', [AdminUserController::class, 'index']);
        Route::post('users/{user}/roles', [AdminUserController::class, 'assignRole']);
    });
});
```

Vistas Blade

- Layout maestro con barra de navegación y sección de mensajes flash.
- Partials reutilizables (navegación, tarjetas de proyecto/tarea, formularios).
- Renderizado condicional por permisos con @can / @role.
- Formularios con @error para mostrar errores de validación campo a campo.

9 Fases de desarrollo

El proyecto se entrega de forma incremental. Cada fase se marca con una etiqueta (tag) en Git, lo que permite evaluar el avance y el uso del control de versiones.

Fase	Contenido	Tag
1 — Cimientos	Migraciones, modelos, relaciones, factories y seeders del dominio.	v0.1
2 — Autenticación	Login/registro por sesión, CSRF, protección de rutas, layout privado.	v0.2
3 — RBAC	spatie: seeder de roles/permisos, asignación, Policiés y control en Blade.	v0.3
4 — CRUD completo	Proyectos, tareas anidadas, comentarios, miembros (N:M), Form Requests, soft delete.	v0.4
5 — Calidad y UX	Filtros, paginación, validaciones afinadas, manejo de errores 403, README.	v1.0

10 Entregables

- Repositorio Git con historial coherente y etiquetas por fase (v0.1 a v1.0).
- Archivo README con instrucciones de instalación, credenciales semilla y tabla de roles/permisos.
- Datos semilla (seeders) que permitan iniciar sesión y probar los cuatro roles desde la interfaz.
- Defensa oral breve del proyecto (opcional, según definición del docente).

11 Criterios de evaluación

La calificación se realiza sobre 100 puntos según la siguiente rúbrica analítica:

Criterio	Qué se evalúa	Pts
Modelo de datos	Relaciones 1:N y N:M correctas, migraciones, factories, seeders, soft deletes.	15
Autenticación	Login/registro por sesión, CSRF, protección de rutas privadas.	20
Roles y permisos	Configuración de spatie, roles y permisos coherentes, asignación funcional.	20
Autorización por pertenencia	Policies correctas y control reflejado en las vistas (@can/@role).	15
CRUD y validación	Operaciones completas con Form Requests y validación robusta.	15
Vistas Blade	Layouts, partials, UX, mensajes flash y paginación.	10
Documentación y Git	README completo y entrega por fases con tags.	5

Escala de niveles de logro

Nivel	Rango	Descripción
Excelente	90 – 100	Cumple todos los requisitos; control de acceso impecable y código limpio.
Bueno	75 – 89	Cumple lo esencial con detalles menores en autorización o validación.
Regular	60 – 74	Funciona parcialmente; fallas en permisos, pertenencia o validación.
Insuficiente	0 – 59	Requisitos incompletos o control de acceso ausente/incorrecto.

12 Recomendaciones técnicas

SUGERENCIAS PARA UN BUEN RESULTADO

Definir primero los permisos y Policies antes que las vistas: el control de acceso debe pensarse desde el diseño, no añadirse al final.

Usar firstOrCreate en los seeders de roles y permisos para que sean idempotentes.

Centralizar el manejo de errores 403 en una vista amigable en lugar de mostrar la traza por defecto.

Mantener los controladores delgados: validar con Form Requests y delegar la autorización a Policies.

Comprometer cambios pequeños y frecuentes en Git; etiquetar cada fase al cerrarla.

— Fin del documento —